

Python数据分析与挖掘所需的Pandas常用知识

原创 2018-01-25 我为峰2014 [Python爱好者社区](#)



点击上方蓝字，快快关注我们哦

作者：我为峰2014

简书专栏：<https://www.jianshu.com/u/clab741ef52e>

前言

Pandas基于两种数据类型：series与dataframe。

一个series是一个一维的数据类型，其中每一个元素都有一个标签。series类似于Numpy中元素带标签的数组。其中，标签可以是数字或者字符串。

一个dataframe是一个二维的表结构。Pandas的dataframe可以存储许多种不同的数据类型，并且每一个坐标轴都有自己的标签。你可以把它想象成一个series的字典项。

Pandas常用知识

一、读取csv文件为dataframe

二、dataframe的数据概况

三、取列数据

四、取行数据

五、取某一单元格数据

六、缺失值处理

七、归一化处理

八、排序

九、索引重新编号

十、求均值

十一、矢量化操作（批量操作）

十二、透视表

一、读取csv文件为dataframe

Pandas很好的一点是，可以操作表文件。输出为dataframe格式，这点很nice。 使用 `pandas.read_csv()` 读取csv文件，输出为dataframe格式数据。 这里数据data.csv数据集下载自百度地图。

```
import pandas as pd
filepath = r'C:/Users/lenovo/Desktop/20180108- 百度地图/20180108-百度地图/data.csv'
df = pd.read_csv(filepath) #为了方便，我只显示三行，其实结果并不是这样子
print(df)
```

检测下数据格式

```
#检测下数据格式是否为DataFrame
print(type(df))
```

```
#输出class 'pandas.core.frame.DataFrame'
```

二、DataFrame数据概况

我们想知道数据如下知识：

- 展示dataframe前后几条记录
- 显示dataframe的列名字
- 查看dataframe的维度情况（几行几列）

2.1展示dataframe前后几行

```
#展示前两条记录（根据需要显示条数） df.head(2) print(df.head(2)) #展示后三条记录 df.tail(3) print(df.tail(3))
```

2.2展示dataframe列名

```
#展示列名 col_names = df.columns print(col_names) #查看下col_names格式 type(col_names) #将col_names转化为list col_list = col_names.tolist() col_list
```

三、从dataframe中取列数据

使用dataframe[column_name]，返回series格式数据。 series序列数据类似于list，你可以近似等同于list。 只不过返回数据中会多一列index索引。如下面的左侧数字序号

3.1 取一列数据

```
#这里我们一列，如取Name列数据 df['Name'][:5] print(df['Name'][:5])
```

3.2取多列数据

```
#这里返回的数据还是dataframe格式，为了方便也只显示前几条记录 cols = ['name', 'province_name', 'city_name', 'city_code', 'area', 'addr'] df[cols] print(df[cols])
```

四、从dataframe中取行数据（记录）

`ix[row, col]` 中括号中第一个参数`row`是行参数，你想选择的数据行数。 第二个参数`col`是列参数，选择你想要的列数据项。

4.1取一行数据

```
#第一行所有数据 df.ix[0, :] print(df.ix[0, :]) #第一行的某几列数据 col =  
['Survived', 'Pclass', 'Sex'] df.ix[0, col] print(df.ix[0, col])
```

2取多行数据

```
#取多行数据，所有列。这里我选择前5行，所有列。 #这里是不是很像切片操作。python  
基础很重要 df.ix[:5, :] print(df.ix[:5, :]) #取多行，某几列 df.ix[:5, col]  
print(df.ix[:5, col])
```

五、取某一单元格数据

取第一行第一列。`df.ix[0,0]` 第三行第七列。`df.ix[2,6]`

六、缺失值处理

缺失值一般标记为NaN, 处理办法如下

```
df.dropna(axis) 默认直接使用df.dropna() axis=1, 按照行进行缺失值处理  
axis=0, 按照列进行缺失值处理 df.dropna(axis=0, subset) axis=0, 按照列方向处  
理subset中的列缺失值 subset=[column] subset含有一个或多个列名的list
```

6.1按照行进行缺失值处理

```
#按照列处理缺失值(为显示方便，只显示前5行) df.dropna(axis=0) #对指定列进行缺  
失值处理 df.dropna(axis=0, subset=['Sex', 'Age'])
```

七、归一化处理

数据集中，不同的列的数据可能在不同量级，如果直接进行分析。模型会认为数字大的影响力大，数字小的影响力小。 最终结果可能导致量级小的变量被剔除出模型。因此需要将数据归一化，变成同一量级的数据，这就是归一化操作。 在这里我们只对一列操作下，其余列也需要操作，但为了方便，这里只写一列的归一化处理。

处理步骤： 1. 选取该列的最大值 `max_value = df[col].max()` 2. 该列所有值均除以 `max_value`

这里要注意，我们会用到pandas特性，矢量化操作，也就是可以对一个列表进行批量同样的操作。

```
#这里我们选Fare列进行归一化，先看下Fare的数据 #为了方便显示，只显示了前10个
df['Fare'] #这里我们选Fare列进行归一化 max_value = df['Fare'].max() max_value
#这里我们选Fare列进行归一化 max_value = df['Fare'].max() max_value #归一化，
并将数据传入新列new_Fare df['new_Fare']=df['Fare']/max_value df['new_Fare']
```

八、排序

```
df.sort_values(col,inplace,ascending) col 对col列进行排序
inplace 布尔型值，是否原地操作。 True时，操作结果覆盖掉原数据，原数据被修改 False时，新建一个新数据，原数据未被修改
ascending 布尔型值。升序降序。 False降序，True升序 #对Age列进行降序操作，不修改原始数据 df.sort_values('Age',inplace=False,ascending=False)
```

九、索引重新

将排序后的索引重新排序 `df.reset_index(drop)` `drop` 为布尔型值，True表示修改原始数据的索引。 False保留原始数据索引序列。
`df.reset_index(drop=False)`

十、求平均值

10.1 所有列的平均值信息

```
df.mean()
```

10.2 单个列的平均值

```
df['Age'].mean()
```

十一、矢量化操作（批量操作）

一般对如list样式的数据批量操作，需要写循环，但是这样费时费力。pandas基于numpy，可进行矢量化操作，一行就能完成复杂的循环语句，而且运行效率还很高。

```
#对Age列批量加10 df['Age']+10).head #对Age列批量减20 df['Age']-10
```

十二、透视表

```
df.pivot_table(index=col1, values=col2, aggfunc='numpy函数')
```

围绕index参数列，分析各个col2，aggfunc是np函数，当然这里的aggfunc也可以是自定义函数。

```
#分析平均年龄对生存率的影响。 #0为死亡，1为生存。 #这里我们发现年龄对生存率有 影 响 。 import numpy as np
df.pivot_table(index='Survived', values='Age', aggfunc=np.mean) #分析仓位等级对生存率影响。0为死亡，1为生存。 #仓位为一等二等三等分别取值1，2，3 #一等舱最高级。我们发现仓位等级对生存也有影响。
df.pivot_table(index='Survived', values='Pclass', aggfunc=np.mean)
```

pandas提取html中的表格数据

pandas会在网页中寻找任何符合html表形式的数据，并将其转化WieDataFrame对象作为返回结果

Code

pandas使用方法

```
import pandas as pd #header=1 显示列名；header=0，不显示
pd.read_html(url,header)
```

实战代码开始

```
import pandas as pd url =
"http://hz.house.ifeng.com/detail/2014_10_28/50087618_1.shtml" data =
pd.read_html(url,header=1) print(data)
```

注意啊，这里得到的数据格式是list。

[		序号	楼盘名称	城区	签约套数	预定套数	签约面积（m²）	
签约均价（元/m²）	0	1.0	龙湖春江郦城	滨江	18	0		
2178.61	23757.0	1	2.0	海威钱塘之星	滨江	13		
0	629.55m²	17398.0	2	3.0	大家运河之星	拱墅		
12	0	1052.72m²	10457.0	3	4.0	保利城市果岭		
下沙	8	0	743.05m²	10457.0	..	...		
...	...	...	...	...	...	85	86.0	
广宇锦绣桃源	拱墅	1	0	86.44m²	12473.0	86	87.0	
	景瑞申花壹号院	拱墅	1	0	89.18m²	21529.0	87	
88.0		复地黄龙和山	西湖	0	1	0m²		
0.0	88	89.0	中粮方圆府	下城	0	1		
0m²		0.0	89	90.0	东方铭楼	下沙	0	16
	0m²	0.0	90	NaN	总计签约：	主城区		
216	40	21755.55m²	NaN	[91 rows x 7 columns],			2	

DataFrame对象

`df.to_json()`

而只要知道数据存储在DataFrame中，一切都变的简单起来。 比如我很希望数据以json记形式输出，很简单！这只是一行代码的事情。

```
import pandas as pd data = pd.read_html(url,header=1) #data数据是list类型，要先转化为dataframe df = pd.DataFrame(data) df.to_json(orient='records')
```

`df.to_csv()`

dataframe对象，还可以将数据输出保存为csv文件

```
import pandas as pd data = pd.read_html(url,header=1) df = pd.DataFrame(data)
#encoding为gbk编码，可以在office excel中看中文不乱吗
df.to_csv('data.csv',encoding='gbk')
```



Python爱好者社区历史文章大合集:

[Python爱好者社区历史文章列表（每周append更新一次）](#)



福利：文末扫码立刻关注公众号，“Python爱好者社区”，开始学习Python课程：

关注后在公众号内回复“课程”即可获取：

小编的Python入门视频课程！！！！

崔老师爬虫实战案例免费学习视频。

丘老师数据科学入门指导免费学习视频。

陈老师[数据分析报告制作](#)免费学习视频。

玩转大数据分析！Spark2. X+Python [精华实战课程](#)免费学习视频。

丘老师[Python网络爬虫实战](#)免费学习视频。

